

SME0230 - Introdução à Programação de Computadores

Primeiro semestre de 2026

Professora: Marina Andretta (andretta@icmc.usp.br)

Monitores: Gabriel Dantas de Oliveira (gabrieloliveira7@usp.br)

Pedro Luis Anghievisck (pedro.anghie@usp.br)

Yan Trindade Meireles (yan.trindade@usp.br)

Trabalho: jogo Scoundrel

1 Grupos

- O trabalho será dividido em duas etapas, que poderão ser realizadas em grupos de até **3** pessoas (é fortemente recomendado que o trabalho seja realizado em trios).
- As duas etapas do trabalho devem ser feitas pelo mesmo grupo.
- Caso alguém deseje mudar de grupo, deverá realizar contato prévio com a professora.

2 Submissão

- As duas etapas do trabalho deverão ser submetidas no e-disciplinas.
- Apenas o arquivo fonte deve ser entregue, no formato `.c`.
- No início do arquivo (`.c`) deverá haver um comentário com o nome e o número USP de todos(as) integrantes do grupo. Caso algum(a) componente não esteja identificado(a), ficará com nota 0 (zero).

Observação: apenas para facilitar a leitura e escrita do texto abaixo, usaremos sempre o gênero masculino.

3 Funcionamento do jogo

O jogo Scoundrel, criado em 2011 por Zach Gage e Kurt Bieg, é um jogo individual, que usa um baralho de cartas comum. A descrição do jogo feita aqui é baseada no texto disponível em <http://stfj.net/art/2011/Scoundrel.pdf> (acessado em 31 de março de 2026).

Se quiser mais informações, incluindo vídeos explicando o funcionamento do jogo, visite <https://boardgamegeek.com/boardgame/191095/scoundrel>.

3.1 Preparação

Para jogar Scoundrel, remova do baralho todos os coringas, figuras (J - valete, Q - dama e K - rei) e A's vermelhos (♦ - ouros e ♥ - copas).

Embaralhe as cartas restantes e coloque o monte virado para baixo à sua esquerda. Esse baralho é chamado de Masmorra (*dungeon*). Você começa com 20 Pontos de Vida.

3.2 Regras

As 26 cartas de naipes pretos do baralho (paus - ♣ e espadas - ♠) são Monstros. O dano de cada um deles é igual ao seu valor numérico. No caso das figuras, valete (J) causa 11 pontos de dano, dama (Q) causa 12, rei (K) causa 13 e ás (A) causa 14.

As 9 cartas de ouros (♦) são Armas. Cada Arma causa dano igual ao seu valor. Cada vez que você decidir pegar uma Arma, você deve equipá-la e sua Arma anterior é descartada.

As 9 cartas de copas (♥) são Poções de Vida. Cada Poção recupera, no máximo, a quantidade de Pontos de Vida correspondente a seu valor. O número de Pontos de Vida está limitado a 20, ou seja, se você tem, por exemplo, 15 Pontos de Vida e usa uma Poção de Vida com valor 7, seus Pontos de Vida atingem o valor máximo de 20 (os 2 Pontos de Vida a mais que você teria se perdem). Você só pode usar uma Poção de Vida por turno. Se você pegar mais de uma Poção de Vida no mesmo turno, a segunda será descartada.

As cartas que são descartadas não são mais usadas, então você pode deixá-las em uma pilha de descarte, com a face virada para baixo.

O jogo termina quando seus Pontos de Vida chegam em 0 (e você perde) ou quando você atravessa toda a Masmorra (você ganha).

3.3 Como jogar

Em cada turno, vire cartas do topo do baralho (Masmorra), uma por uma, até ter 4 cartas viradas para cima à sua frente, formando uma Sala.

Você pode evitar a Sala, se quiser. Caso escolha fazer isso, pegue as quatro cartas de uma só vez e coloque-as no fundo da Masmorra. Embora você possa evitar quantas Salas quiser, não pode evitar duas Salas seguidas. Por isso é bom ter algum marcador informando se você evitou uma Sala.

Se decidir não evitar a Sala, você deve enfrentar ou usar, uma por uma, 3 das 4 cartas que ela contém. Escolha cada uma delas e as resolva separadamente.

- Se você escolher uma Arma (cartas de ouros - ♦): você deve equipá-la. Faça isso colocando-a virada para cima entre você e as cartas restantes da Sala. Se você já tinha uma Arma equipada, descarte-a, juntamente com os Monstros associados a ela, na pilha de descarte.
- Se você escolher uma Poção de Vida (cartas de copas - ♥): adicione o valor dela a seus Pontos de Vida e descarte-a (na pilha de descarte). Lembre-se que seus Pontos de Vida não podem ultrapassar 20 e você não pode usar mais de uma Poção de Vida por turno. Se pegar duas poções no mesmo turno, a segunda é simplesmente descartada.
- Se você escolher um Monstro (cartas de paus - ♣ ou espadas - ♠): você pode enfrentá-lo desarmado ou usando uma Arma equipada. O combate com o Monstro acontece da seguinte forma:
 - Se você escolher lutar contra o Monstro desarmado, subtraia o valor total dele dos seus Pontos de Vida e coloque o Monstro na pilha de descarte.
 - Se você escolher lutar contra o Monstro usando sua Arma equipada, coloque o Monstro virado para cima sobre a Arma (e sobre quaisquer outros Monstros que já estejam nela). Certifique-se de posicionar o Monstro de forma que o valor da Arma ainda fique visível. Subtraia o valor da Arma do valor do Monstro e essa diferença é o que é subtraído dos seus Pontos de Vida.

Por exemplo, se sua Arma equipada tiver valor 5 e você combater um Monstro de valor 3 com ela, não sofre nenhum dano ($3 - 5 < 0$). Por outro lado, se com a mesma Arma de valor 5 você combater um Monstro de valor 12 (dama - Q), você sofre 7 pontos de dano ($12 - 5 = 7$) e perde 7 Pontos de Vida.

É importante observar que, embora você mantenha sua Arma até substituí-la, depois que uma Arma é usada contra um Monstro, ela só pode ser usada para derrotar Monstros de valor menor do que o último Monstro que ela derrotou.

Por exemplo, se sua Arma de valor 5 derrotou um Monstro de valor 12 (dama - Q), você só pode usar essa Arma para combater Monstros com valores menores que 12. O dano da Arma continua sendo de 5, o que muda é somente quais Monstros você pode combater com essa Arma. Suponha que, neste caso, você derrote, com essa Arma, um Monstro de valor 7. Você sofre 2 pontos de dano (já que $7 - 5 = 2$), ou seja, perde 2 Pontos de Vida, e a partir de agora, sua Arma só pode combater Monstros de valor menor que 7.

Se em algum momento do turno seus Pontos de Vida chegam a 0, você perde e o jogo termina.

Caso isso não ocorra, depois de escolher e resolver 3 cartas da Sala (restando apenas uma), seu turno termina. Deixe a quarta carta virada para cima à sua frente como parte da próxima Sala e comece o novo turno.

Se a Masmorra terminar e você ainda tiver Pontos de Vida, você ganha o jogo.

4 Objetivo do trabalho

O objetivo deste trabalho é desenvolver um programa, em linguagem C, que implemente duas versões do jogo Scoundrel: na primeira, uma pessoa joga; na segunda, o próprio computador joga sozinho. A entrega do trabalho é dividida em duas partes. Na primeira, espera-se apenas a versão para uma pessoa jogar. Na segunda, devem ser entregues ambas as versões do jogo.

No início do programa, a pessoa deverá escolher qual das versões será jogada (ou seja, se ela jogará ou se o computador jogará sozinho). Deverá estar claro como a pessoa deve especificar suas jogadas, o que é a Masmorra, a Sala, a Arma que está equipada, os Pontos de Vida atualizados e se quem está jogando pode ou não evitar a Sala atual. Vamos chamar todos esses dados que aparecem para quem está jogando de Mesa.

Na versão em que o computador joga sozinho, espera-se, no mínimo, que o computador seja capaz de realizar jogadas válidas, além de identificar e concretizar vitórias óbvias, ou seja, aquelas que poderiam ser alcançadas no turno atual. E as jogadas feitas por ele devem ser exibidas, para que quem está assistindo possa acompanhar os turnos.

Após a primeira exibição da Mesa, o que pode ser feito automaticamente (como a distribuição das cartas que formam a Sala) deve ser feito e o programa irá aguardar a inserção da instrução de movimento no terminal. A instrução pode ser válida ou não, e uma verificação deverá ser realizada após a inserção dos dados de entrada; caso não seja válida, o programa deverá imprimir uma mensagem de erro e aguardar uma nova inserção.

Em cada inserção de jogada, a representação da Mesa deverá ser atualizada e mostrada na tela. Após cada jogada, o programa deverá verificar se os Pontos de Vida ainda são maiores que 0. Em caso negativo, deve ser informado que quem está jogando perdeu e o jogo é finalizado. Em caso positivo, continua-se o turno ou começa-se um novo.

5 Práticas obrigatórias e proibidas

É necessário que o **código esteja indentado** com tabulação ou até quatro espaços. A indentação é uma prática que torna o código legível e permite uma possível manutenção de maneira mais fácil. Sugere-se a utilização de duas formas comuns de indentação, como mostrado a seguir por meio de exemplos arbitrários na Figura 1.

```
#include <stdio.h>

void imprimir_valor(int n) {
    printf("%d\n", n);
    return;
}

int main() {
    int i = 0;

    while (i < 10) {
        imprimir_valor(i);
        i++;
    }

    return 0;
}
```

```
#include <stdio.h>

void imprimir_valor(int n)
{
    printf("%d\n", n);
    return;
}

int main()
{
    int i = 0;

    while (i < 10)
    {
        imprimir_valor(i);
        i++;
    }

    return 0;
}
```

Figura 1: Exemplos de indentações para o código em linguagem C.

Além disso, é necessário comentar o código. Comentários que apenas descrevem de maneira literal o que o código faz não são muito úteis. São desejáveis comentários que expliquem o que os trechos do programa realizam, facilitando o entendimento por outras pessoas (além dos próprios autores).

Outro aspecto importante é a nomeação de variáveis e funções: devem ser usados nomes significativos. Por exemplo, empregar variáveis com nomes como `a` e `b` para identificar os Pontos de Vida e se uma Sala pode ser evitada dificulta a leitura. Uma opção melhor seria utilizar, por exemplo, `pontos_vida` e `pode_evitar`. Comentários são úteis para auxiliar o entendimento do código, mas não substituem o uso de nomes claros e significativos para variáveis e funções.

A divisão do código em funções será obrigatória apenas na segunda etapa do trabalho. (Se você não sabe o que são funções, não se preocupe: isso será ensinado mais adiante na disciplina.) Caso o grupo opte por utilizá-las na primeira etapa, todas as funções deverão incluir uma instrução `return`, mesmo aquelas declaradas como `void`.

Com o objetivo de promover boas práticas de programação, **é expressamente proibida a utilização dos seguintes recursos de programação**: comando `goto`, comando `continue`, comando `break` e **variáveis globais**. É permitido o uso do comando `break` somente junto ao comando `switch-case`. Se você não sabe o que são estes recursos, pode perguntar à professora e/ou aos monitores como eles funcionam, apenas por curiosidade. Também é proibido: o uso do comando `#define` para definir uma sequência de comandos, a definição de uma função dentro de outra função, o uso de laços com condições que são sempre verdadeiras (como `while (1)` ou `while (x == x)`) e o uso de comandos/recursos que não foram vistos em aula. **O uso de recursos proibidos acarretará em nota 0 (zero) no trabalho.**

Se algum recurso de Inteligência Artificial for usado na implementação, isso deve ser explicitado em um comentário no início do código (tanto que IA foi usada como para que). É permitido o uso de IA para auxiliar em alguns pequenos pontos da implementação, mas não para fazer todo o código. Se acontecer este último caso, **isso acarretará em nota 0 (zero) no trabalho.**

Observa-se que, para toda prática escolhida, é importante *manter o mesmo padrão*. Por exemplo, se você optar por utilizar a abertura das chaves na mesma linha, lembre-se de sempre usá-la dessa forma.

O descumprimento das boas práticas de programação irá acarretar descontos na nota final do trabalho.

6 Prazos de entrega

As duas etapas devem ser entregues no e-disciplinas. As datas e horários máximos de entrega estão especificados lá. Fique atento!